

Proyecto 2 — Agente de juego inteligente para Connect6

Monte Carlo Tree Search (MCTS) con selección UCB1 y aprendizaje por refuerzo profundo en la política de simulación

1. Información de los autores

Cédula de Identidad	Nombre y Apellido
30175126	Jesús Rodolfo Gil
28059781	Gabriel David

2. Descripción del sistema

2.1. Objetivo y reglas del juego

Connect6 se juega sobre un tablero de 19×19 intersecciones. El primer jugador (negras) coloca una ficha en su primer turno y, a partir de entonces, cada jugador coloca dos fichas por turno. Gana quien forme una línea de seis o más fichas propias en sentido horizontal, vertical o diagonal. El objetivo del proyecto es un agente capaz de jugar competitivamente contra un humano u otro agente.

2.2. Arquitectura general del agente

El algoritmo principal es **Monte Carlo Tree Search (MCTS)** (capítulo 6 de Russell & Norvig), potenciado con una **red neuronal profunda entrenada por refuerzo** que actúa como política de simulación, siguiendo la idea del sistema AlphaGo. Sobre esa base se añaden **heurísticas propias de Connect6**. Para cada turno, el agente decide sus fichas con la siguiente prioridad:

1. **Apertura:** si el tablero está vacío, juega al centro (9,9).
2. **Victoria inmediata:** si puede completar seis en línea, lo hace.
3. **Bloqueo:** si el rival amenaza con ganar en su siguiente ficha, lo bloquea.
4. **Búsqueda MCTS:** en cualquier otro caso ejecuta MCTS para elegir la ficha.

El código se organiza en capas: `domain/` (reglas del tablero), `ai/` (heurísticas, MCTS, red neuronal y agente) y `network/` (cliente para la plataforma Connect6 Arena).

2.3. Representación del estado

El tablero se guarda como una matriz 19×19 con valores 0 (vacío), 1 (negras) y 2 (blancas). Para la red neuronal, el estado se codifica desde la óptica del jugador que mueve como un

tensor de forma (19,19,1): 1.0 en sus fichas, -1.0 en las del rival y 0.0 en los vacíos. Esta representación relativa permite que la misma red evalúe a ambos jugadores.

2.4. Algoritmo principal: MCTS

El MCTS construye incrementalmente un árbol de búsqueda repitiendo, en cada simulación, cuatro fases:

1. **Selección:** desde la raíz se desciende eligiendo hijos según la política UCB1 (sección 2.5) hasta llegar a un nodo no expandido del todo.
2. **Expansión:** se añade un nodo hijo correspondiente a una jugada aún no probada.
3. **Simulación (playout):** se juega una partida rápida desde ese nodo hasta un estado terminal (o un límite de profundidad) usando la política de simulación (sección 2.6).
4. **Retropropagación:** el resultado (+1 gana negras, -1 gana blancas, o empate) se propaga hacia la raíz actualizando las visitas N y el valor acumulado w de cada nodo del camino.

Como el turno coloca dos fichas, el estado de búsqueda lleva la cuenta de las fichas que le restan al jugador en el turno; cuando llegan a cero, el turno pasa al rival. Tras todas las simulaciones, el agente juega la ficha del hijo *más visitado* de la raíz (jugada más robusta).

2.5. Política de selección: UCB1 / UCT

La política de selección es **UCB1** (la fórmula de la UCT presentada en la página 209 de Russell & Norvig). Para elegir el hijo se maximiza:

$$UCB1(\text{hijo}) = \frac{W(\text{hijo})}{N(\text{hijo})} + C \cdot \sqrt{\frac{\ln N(\text{padre})}{N(\text{hijo})}}$$

El primer término (explotación) es el valor promedio del nodo; el segundo (exploración) favorece a los nodos poco visitados. La constante C equilibra ambos (se usa $C \approx 1.4 \approx \sqrt{2}$). El valor w se guarda desde la óptica del jugador que movió hacia el nodo, de modo que el signo de la recompensa se ajusta en la retropropagación según ese jugador.

2.6. Política de simulación: red neuronal profunda y aprendizaje por refuerzo

En el MCTS puro, la simulación juega movimientos al azar. La mejora clave de este proyecto — y lo que pide el enunciado— es reemplazar ese azar por la **recomendación de una red neuronal profunda entrenada con aprendizaje por refuerzo**. Así, en la simulación cada jugador coloca la ficha que la red considera mejor.

Arquitectura de la red (DQN). Es una red convolucional (CNN) que recibe el estado (19,19,1) y produce el valor Q de colocar una ficha en cada una de las 361 intersecciones:

- Conv2D 32 filtros (3×3), ReLU, *padding same*.
- Conv2D 64 filtros (3×3), ReLU, *padding same*.
- Flatten → Densa 256 (ReLU) → Densa 361 (salida lineal, un valor Q por casilla).

Aprendizaje por refuerzo (Q-learning profundo). La red se entrena jugando muchas partidas. Los ingredientes son:

- **Recompensa:** +1 al ganar, -1 al perder, o en el resto (recompensa dispersa).
- **Búfer de repetición (replay buffer):** almacena transiciones (estado, acción, recompensa, estado siguiente, fin) y se entrena con mini-lotes aleatorios, rompiendo la correlación temporal.
- **Red objetivo (target network):** copia de la red que se actualiza periódicamente y estabiliza el cálculo del valor objetivo de Bellman $y = r + \gamma \cdot \max_a Q_{\text{target}}(s', a')$.
- **Política ϵ -greedy:** con probabilidad ϵ explora una jugada válida al azar y, si no, explota la mejor según la red. ϵ decae por episodio (de 1.0 hacia 0.1).
- **Optimización:** pérdida de Huber y optimizador Adam.

El modelo entrenado (`dqn_model.keras`) se conecta al MCTS como política de simulación mediante el método `DQNAgent.policy()`.

2.7. Heurísticas y estrategias propias de Connect6

- **Apertura:** primera ficha de negras al centro del tablero (9,9).
- **Victoria inmediata:** detección rápida de la casilla que completa seis en línea.
- **Bloqueo:** detección de la casilla con la que el rival ganaría, para ocuparla.
- **Puntuación táctica:** valora cada casilla por cuántas fichas propias extiende y cuántas líneas del rival corta; guía la simulación cuando no hay red disponible.

2.8. Poda de vecindad (control del factor de ramificación)

Un tablero 19×19 tiene 361 jugadas iniciales, inviable para el MCTS. Por ello el agente sólo considera casillas vacías cercanas (radio 1) a fichas ya colocadas, y adicionalmente restringe la búsqueda a las mejores candidatas según la heurística. Esto reduce el factor de ramificación de 361 a unas pocas decenas, concentrando la búsqueda donde importa.

2.9. Integración con la plataforma Connect6 Arena (gRPC)

El enunciado exige jugar en la plataforma Connect6 Arena, cuyo servidor se comunica con los agentes por medio de la librería gRPC. El agente expone el método `get_action(board, is_first_turn)`, que devuelve la lista de fichas del turno (una en el primer turno de negras, dos en el resto). El módulo `network/client.py` es el punto de integración: traduce los mensajes gRPC del servidor (estado del tablero y turno) a llamadas a `get_action` y envía de vuelta las fichas elegidas.

3. Resultados experimentales

La red se entrenó en Google Colab (GPU) durante 300 episodios contra un agente aleatorio. Luego se evaluó, jugando de negras contra un rival aleatorio, en tres configuraciones:

Agente (juega negras)	Victorias vs. aleatorio
MCTS + red neuronal (agente completo)	100%
MCTS + política de simulación heurística	100%
Red neuronal sola (sin MCTS)	83%

La lectura relevante es la comparación *red sola* (83%) frente a *red dentro del MCTS* (100%): la búsqueda en árbol corrige los errores puntuales de la red, que es justamente el beneficio de combinar MCTS con aprendizaje por refuerzo profundo. Frente a un rival aleatorio la tarea es sencilla y ambas variantes de MCTS alcanzan el 100%; para aislar mejor el aporte de la red convendría enfrentar el agente contra oponentes más fuertes.

La curva de recompensa durante el entrenamiento es ruidosa (valores por episodio entre -1 y $+1$, propio de una recompensa dispersa de ganar/perder), con una media móvil que se mantiene por encima de cero y con picos hacia $+0.5$, lo que evidencia el aprendizaje.

```

      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
0    .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
1    .  .  .  .  .  .  X  O  .  .  .  .  .  .  .  .  .  .  .
2    .  .  .  .  .  .  O  .  .  O  .  .  .  .  .  .  .  .  .
3    .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
4    .  .  O  .  .  .  .  .  .  .  .  .  .  .  .  .  O  .  .
5    .  .  .  .  .  .  .  .  .  .  .  .  O  .  .  .  .  .  .
6    .  O  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .
7    .  .  .  O  .  .  .  .  X  X  .  .  .  .  .  .  .  .  O
8    .  .  .  .  .  .  .  .  X  .  X  .  .  .  .  .  .  .  .
9    .  .  .  .  .  .  .  X  X  X  X  O  .  .  .  .  .  .  O
10   .  .  .  .  .  .  .  X  X  .  O  .  .  .  .  .  .  .  .
11   .  .  .  .  .  .  .  X  X  X  .  .  .  .  .  .  .  .  .
12   .  .  .  .  .  .  .  .  X  .  .  .  .  .  .  .  .  .  .
13   .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .  .

```

Figura 1. Posición final de una partida ganada por el agente (X = negras, O = blancas). Las negras forman una línea vertical de seis fichas en la columna 8 (filas 7 a 12).

Figura de entrenamiento: puede insertarse aquí la gráfica de la curva de recompensa generada por el notebook de Colab (archivo PNG descargable), como Figura 2.

4. Implementación y ejecución

El proyecto está implementado en Python. Estructura principal:

Archivo	Función
<code>domain/board.py</code>	Reglas del tablero y utilidades para la búsqueda.
<code>ai/heuristics.py</code>	Victoria/bloqueo inmediato y puntuación de jugadas.
<code>ai/mcts.py</code>	Motor MCTS con selección UCB1 y playout conectable.
<code>ai/dqn_agent.py</code>	Red neuronal de refuerzo (DQN) usada en el playout.
<code>ai/mcts_agent.py</code>	Agente principal (heurísticas + MCTS + red).
<code>network/client.py</code>	Cliente para Connect6 Arena (gRPC).
<code>main.py</code> / <code>train.py</code>	Partida de demostración / entrenamiento.
<code>Entrenamiento_Connect6_Colab.ipynb</code>	Notebook auto-contenido para entrenar en GPU.

El detalle de ejecución está en `Leeme.txt`. En resumen: `python test_agent.py` prueba la lógica (sin TensorFlow); `python main.py` juega una partida de demostración; y el entrenamiento de la red se realiza en Colab con GPU por su costo computacional.

5. Conclusiones

Se implementó un agente de Connect6 cuyo algoritmo base es MCTS con política de selección UCB1, integrando una red neuronal profunda entrenada por refuerzo como política de simulación, más heurísticas propias del juego. El agente gana el 100% de las partidas evaluadas contra un oponente aleatorio, y los resultados muestran que la búsqueda en árbol mejora las decisiones de la red (de 83% a 100%). El diseño en capas y la política de simulación conectable permiten tanto jugar sin la red (playout heurístico) como reforzar el juego con el modelo entrenado.

6. Referencias bibliográficas

1. Stuart Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4.^a edición, 2022. (MCTS y fórmula UCB1/UCT, cap. 6, pág. 209).
2. David Silver, Aja Huang, Chris J. Maddison, et al. «Mastering the game of Go with deep neural networks and tree search». *Nature*, 529(7587):484–489, 2016.
3. Wikipedia contributors. «Connect6 — Wikipedia, the free encyclopedia», 2022. <https://en.wikipedia.org/wiki/Connect6>
4. Laura Parilli y Miguel Salomón. «Connect6 Arena» (starter-kit), 2026. <https://github.com/lauraparilli/connect6-starter-kit>
5. gRPC. Librería de comunicación de alto rendimiento. <https://grpc.io/>